

RELATÓRIO – SISTEMA DE CHAT EM TEMPO REAL

Autores: Nícolas de Freitas Silva, Nicolas de Godoi Stefankowski

Disciplina: Programação orientada a objetos II – Projeto Final

Data: Julho de 2026

1. INTRODUÇÃO

O presente projeto consiste no desenvolvimento de um sistema de comunicação instantânea, baseado em arquitetura cliente-servidor, implementado inteiramente na linguagem Python. O sistema permite a troca de mensagens em tempo real entre múltiplos usuários, oferecendo funcionalidades completas de autenticação, gerenciamento de conversas, envio e recebimento de mensagens, além da persistência de todos os dados em banco de dados SQLite.

A motivação para o desenvolvimento deste sistema reside na necessidade de compreender e aplicar conceitos fundamentais de redes de computadores, programação concorrente, desenvolvimento de interfaces gráficas e gerenciamento de dados. O projeto aborda desde a comunicação em baixo nível com sockets TCP até a implementação de uma interface amigável utilizando a biblioteca Tkinter, passando por camadas intermediárias de serviços e repositórios que garantem a organização e manutenibilidade do código.

Este relatório técnico apresenta uma análise aprofundada da estrutura do sistema, sua arquitetura, os protocolos de comunicação estabelecidos, os principais componentes implementados, bem como uma avaliação crítica com sugestões de melhorias e evolução do sistema.

2. ARQUITETURA DO SISTEMA

2.1 Estrutura Geral

O sistema foi desenvolvido seguindo uma arquitetura cliente-servidor, com clara separação de responsabilidades em camadas bem definidas. A comunicação entre as partes ocorre por meio de conexões TCP estabelecidas via sockets, garantindo entrega confiável das mensagens.

A arquitetura adotada segue o padrão de camadas, com a seguinte organização hierárquica:

Camada de Apresentação: Responsável pela interface com o usuário, implementada em Tkinter, contendo todas as telas e elementos visuais que permitem ao usuário interagir com o sistema.

Camada de Comunicação: Gerencia a conexão socket, o envio e recebimento de mensagens entre cliente e servidor, incluindo o tratamento de threads para operações assíncronas.

Camada de Serviços: Contém a lógica de negócio, incluindo os serviços de autenticação e gerenciamento de conversas.

Camada de Repositórios: Responsável pelo acesso e manipulação dos dados persistidos, abstraindo as operações de banco de dados.

Camada de Persistência: Gerencia a conexão com o banco de dados SQLite e a criação das estruturas de armazenamento.

2.2 Componentes do Servidor

O servidor é composto por três elementos principais que trabalham em conjunto para atender as requisições dos clientes.

O Servidor Socket atua como ponto central de escuta, aceitando novas conexões de clientes e delegando o tratamento de cada um a uma thread dedicada. Esta abordagem permite que múltiplos usuários estejam simultaneamente conectados e ativos.

O Cliente Handler é a peça fundamental do servidor. Cada instância deste componente é executada em uma thread separada e é responsável por processar todas as mensagens recebidas de um cliente específico. Ele mantém o estado da sessão do usuário, verifica sua autenticação, processa os comandos recebidos e encaminha mensagens para outros usuários quando necessário.

O Gerenciador de Usuários Online mantém um mapeamento entre os identificadores dos usuários e seus respectivos sockets, permitindo o roteamento eficiente de mensagens para destinatários que estejam ativos no momento do envio.

2.3 Componentes do Cliente

O cliente gráfico, implementado em Tkinter, foi estruturado em seis telas principais, cada uma correspondendo a uma funcionalidade específica do sistema.

A Tela Inicial apresenta as opções de login e registro, servindo como ponto de entrada para usuários novos e existentes.

A Tela de Login contém os campos para usuário e senha, além de mecanismos para exibir feedback visual sobre o sucesso ou falha na autenticação.

A Tela de Registro permite a criação de novas contas, com validações em tempo real para garantir a qualidade dos dados fornecidos, incluindo força da senha e validação do CEP através de integração com API externa.

A Tela de Lista de Conversas exibe todas as conversas ativas do usuário, permitindo navegação rápida e acesso ao histórico de mensagens.

A Tela de Chat é o núcleo da comunicação, exibindo o histórico de mensagens em formato de balões, com distinção visual entre mensagens enviadas e recebidas, além da área para composição e envio de novas mensagens.

A Tela de Perfil possibilita a edição dos dados cadastrais e a exclusão da conta, com atualização automática das informações.

2.4 Modelo de Dados

O sistema utiliza três entidades principais para representar os dados persistentes. A entidade Usuário armazena todas as informações pessoais, incluindo identificador, nome completo, nome de usuário único, e-mail único, hash da senha e CEP. A entidade Conversa representa um canal de comunicação entre dois usuários específicos, contendo os identificadores de ambos os participantes. A entidade Mensagem registra cada mensagem

trocada, associando-a a uma conversa, indicando o remetente, o conteúdo textual e a data e hora do envio.

3. PROTOCOLO DE COMUNICAÇÃO

3.1 Estrutura das Mensagens

O sistema utiliza um protocolo textual simples, baseado no envio de mensagens estruturadas com delimitadores específicos. Cada mensagem segue o formato geral: `COMANDO;AÇÃO;PARÂMETRO1;PARÂMETRO2;...`

Este formato foi escolhido por sua simplicidade de implementação e facilidade de depuração, permitindo que os desenvolvedores visualizem o conteúdo das mensagens durante testes e manutenção.

3.2 Comandos e Ações

O protocolo define comandos categorizados por área de atuação. O comando AUTH trata de questões relacionadas à autenticação e registro de usuários, incluindo as ações LOGIN, REGISTER e suas respectivas respostas de sucesso ou erro. O comando CHAT gerencia toda a comunicação entre usuários, abrangendo o envio de mensagens, listagem de conversas, abertura de canais e recebimento de novas mensagens. O comando USER permite a gestão do perfil do usuário, com ações para atualização e exclusão de conta. Mensagens de controle como CTRL são utilizadas para indicar erros, confirmações de sucesso e finalização de listagens.

3.3 Fluxos de Comunicação

O processo de autenticação segue um fluxo bem definido. O cliente envia suas credenciais ao servidor, que verifica a existência do usuário no banco de dados e a validade da senha utilizando bcrypt. Em caso de sucesso, o servidor retorna os dados completos do usuário e o registra como online. Falhas são sinalizadas com mensagens de erro apropriadas.

O envio de mensagens segue um fluxo síncrono em que o cliente envia a mensagem com o identificador do destinatário, o servidor persiste a mensagem no banco de dados e a reencaminha imediatamente se o destinatário estiver online. Mensagens enviadas para usuários offline são armazenadas e entregues quando o destinatário se conectar e solicitar o histórico de conversas.

A listagem de conversas envolve uma operação mais complexa. O servidor consulta todas as conversas do usuário, identifica os outros participantes e envia uma mensagem para cada conversa encontrada, finalizando com um marcador de fim de lista.

4. ASPECTOS TÉCNICOS RELEVANTES

4.1 Segurança

A segurança foi considerada em diversos aspectos do sistema. As senhas são armazenadas utilizando o algoritmo bcrypt com salts aleatórios, garantindo que mesmo em

caso de vazamento de dados, as senhas não possam ser facilmente recuperadas. A validação de senhas fortes no momento do registro exige um mínimo de oito caracteres, incluindo letras maiúsculas e minúsculas, números e caracteres especiais. A integração com a API ViaCEP permite validar endereços brasileiros, garantindo que apenas CEPs reais sejam aceitos no cadastro.

4.2 Concorrência

O servidor utiliza múltiplas threads para atender vários clientes simultaneamente. Cada conexão recebe sua própria thread, permitindo que o sistema escale para atender múltiplos usuários. O gerenciamento dos usuários online utiliza um dicionário compartilhado entre threads, que deve ser considerado em futuras implementações para garantir acesso seguro em ambientes com alta concorrência.

4.3 Interface Gráfica

A interface foi desenvolvida com ênfase na experiência do usuário, utilizando cores suaves para diferentes elementos da interface. Mensagens próprias aparecem com um fundo verde claro, enquanto mensagens recebidas utilizam fundo branco, proporcionando distinção visual imediata.

A separação do código da interface em classes independentes para cada tela facilita a manutenção e permite a substituição do toolkit gráfico, se necessário.

5. ANÁLISE CRÍTICA E RECOMENDAÇÕES

5.1 Pontos Fortes

A arquitetura em camadas demonstra uma preocupação clara com a organização e manutenibilidade do código, permitindo que cada componente seja desenvolvido e testado de forma independente. A escolha de SQLite como banco de dados simplifica a implantação, eliminando a necessidade de configuração de um servidor de banco de dados separado.

A implementação da interface gráfica com Tkinter oferece uma experiência satisfatória ao usuário final, com feedback visual adequado para ações críticas como login, registro e envio de mensagens.

5.2 Limitações Identificadas

A ausência de criptografia no canal de comunicação expõe os dados trafegados a interceptação, representando um risco significativo em redes não confiáveis. A limitação do buffer de recepção a 1024 bytes pode ser insuficiente para mensagens longas ou futuras implementações de envio de arquivos.

O uso de threads pode se tornar um gargalo à medida que o número de usuários cresce, pois cada conexão consome recursos do sistema operacional. Não há mecanismo de detecção de clientes desconectados, o que pode levar ao acúmulo de threads inativas.

5.3 Sugestões de Melhorias

A implementação de SSL/TLS é recomendada para garantir a confidencialidade e integridade dos dados transmitidos. A adoção de um sistema de heartbeats ou keepalives permitiria detectar desconexões de forma mais eficiente.

Para melhor escalabilidade, o servidor poderia ser adaptado para utilizar o módulo asyncio, que oferece concorrência mais eficiente baseada em event loops, reduzindo o overhead de threads.

A estruturação de logs em níveis permitiria um diagnóstico mais preciso de problemas em produção, enquanto a adição de testes automatizados aumentaria a confiabilidade do sistema.

6. CONCLUSÃO

O sistema de chat em tempo real desenvolvido representa uma implementação completa e funcional de uma aplicação cliente-servidor, abrangendo desde a comunicação em baixo nível com sockets até a interface gráfica com o usuário. A arquitetura adotada demonstra maturidade no design de software, com clara separação de responsabilidades e organização em camadas.

As funcionalidades implementadas atendem aos requisitos básicos de um sistema de mensagens, incluindo autenticação segura, gerenciamento de conversas, envio em tempo real e persistência de dados. A interface gráfica proporciona uma experiência de usuário agradável e intuitiva.

Embora existam limitações e oportunidades de melhoria, a base do sistema é sólida e pode ser facilmente expandida para incorporar novas funcionalidades e atender a requisitos mais exigentes. O conhecimento adquirido durante o desenvolvimento deste projeto abrange conceitos fundamentais de redes, programação concorrente e desenvolvimento de aplicações completas, representando um valioso aprendizado prático.